Predictive Memory Paging System with Workload Awareness

Spoken script for the faculty project review · about 15 minutes, then questions

G.K. Shreyas Bharadwaj (24BCE2256) · Vaastav Bajanthri (24BCE2223) · P. Giphy Bershan (24BCE2201)

Normal text is spoken aloud. *Amber italic lines are stage directions*. Before you start, open the slide deck, a terminal in the project folder, and **results/report.html** in a browser.

1. Opening (1 min)

Slide 1 on screen.

Good morning. Our project is the **Predictive Memory Paging System with Workload Awareness**.

In one sentence: we built a simulator of operating-system paging, and used it to test whether predicting the next pages a program needs, and loading them early, reduces page faults. The key part is a decision engine that prefetches only when it measurably helps.

Our headline result: on 22 real benchmark traces, the final system removes **55% of the page faults** that standard LRU suffers, against 51% for Linux-style readahead. It is worse than LRU in only 1 of 44 tests. In estimated running time it ties readahead, and I will be upfront about that when we get to the results.

I will cover the problem, how the system works, how we tested it, the results, and what is and is not new.

2. The problem (2 min)

Slide: RAM versus disk.

Every program keeps its data in pages of 4 KB. RAM is fast but small; the disk is large but thousands of times slower. When a program asks for a page that is not in RAM, it must stop and wait while the page is read from disk. That wait is a **page fault**.

Traditional operating systems are *reactive*. They wait for the fault, fetch the page, and if RAM is full, an eviction policy such as FIFO, LRU or Clock decides which page to throw out.

Our idea is to be *proactive*: predict the next pages and bring them in **before** they are asked for. This is called **prefetching**. If the guess is right, the fault never happens.

But here is the catch, and it is the heart of the project. When RAM is full, every prefetched page forces another page out. A wrong guess wastes a disk read, and it may evict a page the program needed soon, creating a fault that would never have happened. So a careless prefetcher can make the system **worse** than doing nothing.

Think of a student at a small desk in a large library. The desk is RAM and the shelves are the disk. A helper who brings the right books saves trips to the shelves. A helper who guesses badly fills the desk with useless books and pushes away the ones the student was reading.

So the project needs two things: a **predictor** that guesses, and a **decision engine** that decides whether each guess is worth the risk and learns from whether prefetching is actually helping.

3. The core idea (1.5 min)

Slide: prediction on top of eviction.

The key design choice is that prediction runs **on top of** an eviction policy. The predictor decides what arrives early; the eviction policy, LRU in our final system, decides what leaves. We never replace LRU; we add to it.

We tried several predictors and ended with a combination of two:

1. **Readahead**, the method Linux uses. When the program reads two consecutive pages, it starts a stream and fetches a window ahead. The window doubles each time the program reaches its middle, for example 4, then 8, then 16 pages, up to a quarter of RAM, and resets when the sequence breaks.
2. **Delta-Markov.** A Markov chain learns "after this, what usually comes next". Ours remembers the *gaps* between pages instead of the page numbers, so a repeating pattern such as +1, +3 lets it predict pages it has never seen. Its tables have 256 entries, so each prediction is a single lookup.

We started with a plain Markov predictor on page numbers, following Joseph and Grunwald, 1997. On its own it removed only about 8% of faults on average. That taught us an important lesson: **prediction accuracy and fault reduction can rank predictors in almost opposite orders.** Readahead's top guess is exactly right only 2.6% of the time, yet it avoids the most faults of any single predictor, because it fetches pages the program has never touched. So we judge every change by page faults in the full system, never by accuracy alone.

4. System architecture (1.5 min)

Slide: architecture diagram. Point at each box as you name it.

The whole system is a simulator written in Python. We record the sequence of pages a real program asked for, called a **trace**, and replay it through software models of RAM, eviction policies, predictors and decision engines. A simulator gives every policy exactly the same input, makes results reproducible, and lets us compute Belady's OPT, which needs to know the future.

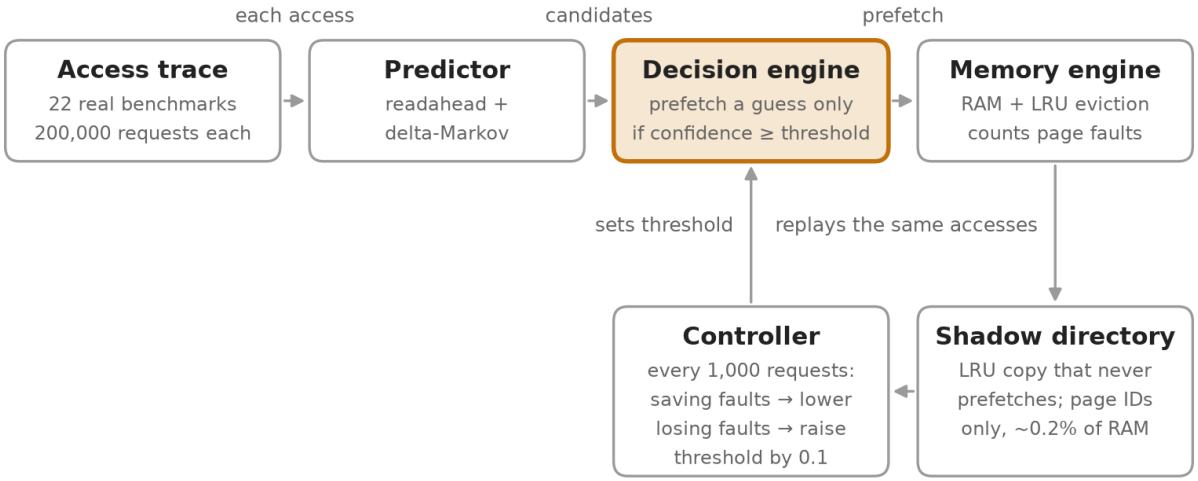


Figure: one access flows left to right; the shadow directory and controller form the feedback loop.

Follow one access from left to right. The trace feeds the predictor, which proposes candidate pages with a confidence. The decision engine accepts or rejects each one. Accepted pages enter RAM through one single function in the memory engine, so a broken predictor can slow the system down but can never make it wrong. Underneath, the shadow directory and the controller form the feedback loop.

In the code, **engine.py** holds RAM and the eviction policies, **predictors.py** the predictors, **deciders.py** the decision engines, **simulate.py** the replay loops, **timing.py** the time model, and **final.py** the frozen final system.

5. Decision engine and feedback (2 min)

Slide: decision rule and the three feedback signals.

The decision engine follows a simple rule:

- If RAM has a **free frame**, prefetch when confidence is at least 0.15, because a wrong guess then evicts nothing.
- If RAM is **full**, prefetch only when confidence is at least an adaptive threshold between 0.15 and 0.95.

The real question is how that threshold should move. We tested three feedback signals, and this is our main finding.

1. **The predictor's own confidence** cannot do it. The predictor keeps two successors per context, so the top one always holds at least half the counts. Confidence never falls below 0.5, and a 0.3 threshold behaves exactly like "always prefetch". This is partly built into the design, and we say so.
2. **Prefetch precision**, the share of prefetched pages that get used, is misleading. On one test workload precision was 81 to 86% while faults went *up*, because a prefetch that is used can still have evicted a page needed sooner.

3 **Fault delta** works: faults without prefetching minus faults with prefetching. It measures exactly what we care about.

But how do we know the faults *without* prefetching while we are prefetching? That is the **shadow directory**: a second copy of LRU that never prefetches and stores page IDs only, no page data, which costs about 0.2% of RAM. Every 1,000 requests the controller compares the two fault counts. If prefetching saved more than 0.002 faults per request, the threshold drops by 0.1; if it cost more than that, the threshold rises by 0.1; otherwise it stays. Tracking a hash sample of just 1 page in 8 gave identical fault counts on 9 of the 10 traces we tested.

6. Experimental setup (1.5 min)

Slide: setup.

- **Data.** 22 real traces from the ML-DPC LoadTraces set: SPEC CPU 2006 programs such as bwaves, leslie3d, mcf and omnetpp, and GAP graph benchmarks such as BFS, PageRank and SSSP. Each has 200,000 requests, mapped to 4 KB pages. Six synthetic traces are used as correctness tests.
- **Memory size.** RAM is set to 1% and 5% of each trace's working set, so 22 traces times 2 sizes gives **44 configurations**. We never use a fixed frame count, because with plenty of memory every policy gives the same result.
- **Baselines.** FIFO, LRU, Clock, Belady's OPT, ARC, LRU-2, and Linux-style readahead.
- **Metric.** vs_LRU equals LRU faults minus our faults, divided by LRU faults. Positive is better; negative means prefetching created extra faults.
- **No tuning on test data.** Every setting was chosen on the first 25% of each trace only, with the selection rule written down before each sweep, and then frozen.
- **Correctness.** 24 automated tests pass. They check that no eviction policy beats OPT, that more memory never adds faults for LRU or OPT, and that every new feature switched off reproduces the old numbers exactly. Independent implementations of LRU, FIFO and OPT gave zero mismatches on 40 traces.

We also built a **time model**, because fewer faults does not automatically mean faster. It replays the I/O timeline with stated latencies: 80 microseconds per read on an NVMe SSD with 8 parallel channels, and 5 milliseconds on a hard disk with one channel. These are model estimates, not hardware measurements.

7. Results (2.5 min)

Slide: results table. Open results/report.html if asked for detail.

Our final system removes the most page faults of every system we tested, and in estimated time it ties Linux readahead.

System	Faults removed vs LRU (mean of 44)	Worst configuration	Speed-up vs LRU (NVMe model)

Final: readahead + delta-Markov, shadow feedback	**55.0%**	–3.3%	1.43×
Linux-style readahead	50.7%	–19.3%	1.44×
Markov prefetching alone	7.7%		0.97×

Speed-up assumes 1 microsecond of compute per access, prediction cost included; geometric mean over 22 traces. Model estimates, not measurements.

What to take from this:

1. **Fewer faults.** The final system has fewer faults than readahead in 34 of 44 configurations, equal in 8, and more in 2. It is worse than LRU in only 1 of 44, by 3.3%; readahead's worst case is 19.3% worse than LRU.
2. **It beats OPT in 30 of 44 configurations.** That sounds impossible, but OPT is only optimal at choosing what to *evict*. It still pays a fault on the first touch of every page, and prefetching avoids those first-touch faults.
3. **The workload matters.** On leslie3d at 1% memory we remove 46.9% of faults against readahead's 24.1%. On graph workloads the gains are smaller, and on lbm nothing is predictable, so every system matches LRU.
4. **Time is the honest part.** On the NVMe SSD model we are 1.43 times faster than LRU, level with readahead at 1.44. The extra faults we remove cost extra prefetch reads. On a single-channel hard disk every prefetcher is slower than LRU, ours the most (0.70 against 0.80), because each prefetch costs as much as the fault it saves.
5. **Timeliness beats accuracy.** Markov alone removes 35 to 37% of faults on leslie3d and bwaves, but it is not faster: it prefetches only one request ahead, so its pages often arrive after they are needed.

Finally, the decision layer is what makes prefetching safe. "Always prefetch" with the same predictor gives the same average benefit, but on our synthetic adversarial test it adds 21.5% extra faults. Our decider cuts that to 2.4%.

8. What is new, and limitations **(1.5 min)**

Slide: contribution and limitations. Say this before you are asked.

We want to be clear about what is ours. The building blocks are known: Markov prefetching is Joseph and Grunwald, 1997; adapting to the fault rate resembles the classic Page Fault Frequency method; readahead is in Linux; and ghost lists come from ARC. Our contributions are:

1. An evaluation of **which feedback signal** a prefetcher should use, and why confidence and precision fail.
2. An exact and cheap way to measure that signal: the **shadow directory**, which stores page IDs only and can be sampled down to 1 page in 8.
3. The **readahead plus delta-Markov** combination under that feedback.
4. A **time-aware** evaluation, which showed that fault counts alone overstate the benefit.

And our limitations:

- The traces are cache-level (LLC) traces treated as pages. That approximates OS paging; it is not real OS paging.
- The time results come from a model with stated latencies, not hardware. The model gives demand reads no priority, which is pessimistic for prefetching.
- Our predictor sees every memory request, while a real kernel only sees page faults.
- Each configuration was run once. The traces are deterministic, so repeats are identical; robustness comes from 22 traces and two memory sizes instead.

9. Demo (2 min)

Switch to the terminal in the project folder. Run the tests first: they take about 13 seconds.

Let me show it running. First, the automated tests, which check the invariants I mentioned and pin the numbers from our real runs:

```
pytest -q
```

Point at the result: all tests pass. If any pinned number changed, it would mean the behaviour changed.

Next, the final report, which collects every table and figure from our runs:

```
Open results/report.html in the browser
```

Scroll to the main results table and the time-model table. Here you can see each trace, the faults for every system, and the estimated speed-up. The final system itself is one function, **run_final** in **src/paging/final.py**, and its settings are frozen in **config.py**, so nobody can quietly retune it on the evaluation data.

10. Conclusion and future work (1 min)

Back to the final slide.

To conclude. Prefetching can remove more than half of LRU's page faults, but only if something decides when to trust the predictor. The right signal for that decision is the measured fault delta, not the predictor's confidence and not prefetch precision, and a shadow directory of page IDs can measure it cheaply. Combined with readahead and a delta-Markov predictor, our system removes 55% of LRU's faults with the best worst case of every system we tested. In time it matches Linux readahead rather than beating it, and we report that as it is.

Next, we would evaluate on real OS page-fault traces or inside a kernel, give demand reads priority over prefetches, control how far ahead to prefetch, and compare against a learned predictor under the same time model.

Thank you. We are happy to take questions.

Appendix: likely questions and short answers

Every number comes from the project's result files (results/*.csv and docs/RESULTS.md).

How can your system beat OPT, the optimal policy?

OPT is optimal only at choosing what to evict. It still pays for the first access to every page. Prefetching avoids those first-access faults. The final system has fewer faults than OPT in 30 of 44 configurations.

Why a simulator and not a real operating system?

Every policy sees exactly the same input, results are reproducible, and OPT, which needs the future, can be computed. Changing a kernel would take far more engineering than a course project allows.

If readahead ties you in time, why not just use readahead?

Our system removes more faults (fewer than readahead in 34 of 44 configurations), helps much more on workloads like leslie3d (46.9% against 24.1% at 1% memory), and has a far better worst case (–3.3% against –19.3%).

Why does prefetching lose on a hard disk?

A hard disk serves one read at a time, so each prefetch costs as much as the fault it tries to save.

Why is Markov alone not faster, although it removes 35 to 37% of faults on leslie3d and bwaves?

It prefetches only one request ahead, so its pages often arrive after they are needed.

Isn't the shadow directory just an impossible oracle?

It holds page IDs only, no page data, about 0.2% of RAM, and can be sampled down to 1 page in 8 with identical fault counts on 9 of 10 traces tested.

Why not deep learning?

The predictor runs on every memory request, so it must be constant-time and tiny; ours uses 256-entry tables. A learned predictor is a fair comparison for future work.

How did you avoid tuning on test data?

Every setting was chosen on the first 25% of each trace only, with the selection rule written down before each sweep, and then frozen in config.py.

How do you know the simulator is correct?

24 automated tests check invariants, equivalences and pinned numbers. Independent implementations of LRU, FIFO and OPT gave zero mismatches on 40 traces, and a rerun reproduced all 48 rows of the original Colab results exactly.

Why so little gain on graph workloads?

About 99.8% of predictions on BFS name a page that is already in RAM, so there is almost nothing left to prefetch.

Did ARC or LRU-2 help?

No. ARC averaged –0.5% against LRU on our traces, and LRU-2 was much worse on leslie3d and bwaves.

Is this publishable?

We have a paper draft for a student or regional conference, framed as an evaluation study. We do not claim the components are new.